

Rapid Web Development with Python/Django

URLs and Views

B. Wakim

Outline

- | | |
|--|--|
| <ul style="list-style-type: none">• URLs<ul style="list-style-type: none">➤ URL dispatcher➤ URL conf.➤ Path converters➤ Using regular expressions in path➤ Passing extra options to view functions➤ Including other URLconfs | <ul style="list-style-type: none">• Views<ul style="list-style-type: none">➤ Writing views➤ HttpResponse➤ HttpRequest• Django shortcut functions<ul style="list-style-type: none">➤ Render➤ Redirect• References |
|--|--|

URLs

URL dispatcher

- To design URLs for an app, you create a Python module informally called a `URLconf` (URL configuration). This module is pure Python code and is a mapping between `URL path` expressions to Python functions (your `views`).
- **How Django processes a request**
- When a user requests a page from your Django-powered site, this is the algorithm the system follows to determine which Python code to execute:
 - Django determines the root `URLconf` module to use.
 - Django loads that Python module and looks for the variable `urlpatterns`. This should be a sequence of `django.urls.path()` and/or `django.urls.re_path()` instances.
 - Django runs through each URL pattern, in order, and stops at the first one that matches the requested URL, matching against `path_info`.
 - If no URL pattern matches, or if an exception is raised during any point in this process, Django invokes an appropriate error-handling view.

Example URL conf.

```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [  
    path('articles/2003/', views.special_case_2003),  
    path('articles/<int:year>/', views.year_archive),  
    path('articles/<int:year>/<int:month>/', views.month_archive),  
    path('articles/<int:year>/<int:month>/<slug:slug>/', views.article_detail),  
]
```

- **Notes:**
- To capture a value from the URL, use angle brackets.
- Captured values can optionally include a *converter* type. For example, use `<int:name>` to capture an integer parameter. If a converter isn't included, any string, excluding a `/` character, is matched.
- There's no need to add a **leading** slash, because every URL has that.

Path converters¶

- The following path converters are available by default:
 - **str** - Matches any non-empty string, excluding the path separator, '/'. This is the default if a converter isn't included in the expression.
 - **int** - Matches zero or any positive integer. Returns an int.
 - **slug** - Matches any slug string consisting of ASCII letters or numbers, plus the hyphen and underscore characters. For example, *building-your-1st-django-site*.
 - **path** - Matches any non-empty string, including the path separator, '/'. This allows you to match against a complete URL path rather than a segment of a URL path as with str.

Example URL conf.

- **Example requests:**
 1. `path('articles/2003/', views.special_case_2003),`
 2. `path('articles/<int:year>/', views.year_archive),`
 3. `path('articles/<int:year>/<int:month>/', views.month_archive),`
 4. `path('articles/<int:year>/<int:month>/<slug:slug>/', views.article_detail),`
- A request to `/articles/2005/03/` would match the third entry in the list. ⇔ call the function `views.month_archive(request, year=2005, month=3)`.
- `/articles/2003/` would match the first pattern in the list ⇔ call the function `views.special_case_2003(request)`
- `/articles/2003` would not match any of these patterns, because each pattern requires that the URL end with a slash.
- `/articles/2003/03/building-a-django-site/` would match the final pattern.
⇔ call the function `views.article_detail(request, year=2003, month=3, slug="building-a-django-site")`.

What the URLconf searches against¶

- The URLconf searches against the requested URL, as a normal Python string. This does not include GET or POST parameters, or the domain name.
- For example, in a request to <https://www.example.com/myapp/>, the URLconf will look for [myapp/](#).
- In a request to <https://www.example.com/myapp/?page=3>, the URLconf will look for [myapp/](#).
- The URLconf doesn't look at the request method. In other words, all request methods – POST, GET, HEAD, etc. – will be routed to the same function for the same URL.

Using regular expressions in path¶

- To use regular expression , use `re_path()` instead of `path()`.
- In Python regular expressions, the syntax for named regular expression groups is `(?P<name>pattern)`, where name is the name of the group and pattern is some pattern to match.

```
from django.urls import path, re_path
from . import views
urlpatterns = [
    path('articles/2003/', views.special_case_2003),
    re_path(r'^articles/(?P<year>[0-9]{4})/$', views.year_archive),
    re_path(r'^articles/(?P<year>[0-9]{4})/(?P<month>[0-9]{2})/$', views.month_archive),
    re_path(r'^articles/(?P<year>[0-9]{4})/(?P<month>[0-9]{2})/(?P<slug>[\w-]+)/$',
views.article_detail),
]
```

- The exact URLs that will match are slightly more constrained. For example, the year 10000 will no longer match since the year integers are constrained to be exactly four digits long.
- Each captured argument is sent to the view as a string, regardless of what sort of match the regular expression makes.

Regular Expressions (RE)

- A regular expression (or RE) specifies a set of strings that matches it.
- **Some useful REs in Python**
 - Special characters: '.', '^', '\$', '*', '+', '?', '\\', '|'
 - Brackets: {m}, {m, n}, [ab], [0-9], (ab)
 - Others: (?P<name>...), \\d, \\s
 - In the Site field, use substrings with special characters — * and ? .
 - An asterisk (*) stands for any number of characters,
 - a question mark (?) represents one character.

Regular expression Symbol (Meaning)

<code>^</code> (Start of url)	<code>\$</code> (End of url)	<code>\</code> (Escape for interpreted values)	<code> </code> (Or)
<code>+</code> (1 or more occurrences)	<code>?</code> (0 or 1 occurrences)	<code>{n}</code> (n occurrences)	<code>{n,m}</code> (Between n and m occurrences)
<code>[]</code> (Character grouping)	<code>(?P___)</code> (Capture occurrence that matches regexp ___ and assign it to name)	<code>.</code> (Any character)	<code>\d+</code> (One or more digits). Note escape, without escape matches <code>'d+'</code> literally]
<code>\D+</code> (One or more non-digits).Note escape, without escape matches <code>'D+'</code> literally]	<code>[a-zA-Z0-9_]+</code> (One or more word characters, letter lower-upper case, number or underscore)	<code>\w+</code> (One or more word characters, equivalent to <code>[a-zA-Z0-9_]</code>) Note escape, without escape matches <code>'w+'</code> literally]	<code>[-@\w]+</code> (One or more word character, dash or at sign). Note no escape for <code>\w</code> since it's enclosed in brackets (i.e. a grouping)

Regular expression Symbol (Meaning)

Url regular expression	Description	Sample urls
<code>re_path(r'^\$',.....)</code>	Empty string (Home page)	Matches: <code>http://127.0.0.1/</code>
<code>re_path(r'^stores/',.....)</code>	Any trailing characters	Matches: <code>http://127.0.0.1/stores/</code> <code>http://127.0.0.1/stores/long+string+with+12345</code>
<code>re_path(r'^about/contact/\$',.....)</code>	Exact, no trailing characters	Matches: <code>http://127.0.0.1/about/contact/</code> Doesn't match: <code>http://127.0.0.1/about/</code>
<code>re_path(r'^stores/\d+/',.....)</code>	Number	Matches: <code>http://127.0.0.1/stores/2/</code> <code>http://127.0.0.1/stores/34/</code> Doesn't match: <code>http://127.0.0.1/stores/downtown/</code>
<code>re_path(r'^drinks/\D+/',.....)</code>	Non-digits	Matches: <code>http://127.0.0.1/drinks/mocha/</code> Doesn't match: <code>http://127.0.0.1/drinks/324/</code>
<code>re_path(r'^drinks/mocha espresso/',.....)</code>	Word options, any trailing characters	Matches: <code>http://127.0.0.1/drinks/mocha/</code> <code>http://127.0.0.1/drinks/mochaccino/</code> <code>http://127.0.0.1/drinks/espresso/</code> Doesn't match: <code>http://127.0.0.1/drinks/soda/</code>
<code>re_path(r'^drinks/mocha\$ espresso/\$',.....)</code>	Word options exact, no trailing characters	Matches: <code>http://127.0.0.1/drinks/mocha/</code> Doesn't match: <code>http://127.0.0.1/drinks/mochaccino/</code> Matches: <code>http://127.0.0.1/drinks/espresso/</code>

Passing extra options to view functions

- RLconfs have a hook that lets you pass extra arguments to your view functions, as a Python dictionary.
- The `path()` function can take an optional third argument which should be a dictionary of extra keyword arguments to pass to the view function.
- For example:

```
from django.urls import path
from . import views
urlpatterns = [
    path('blog/<int:year>/', views.year_archive, {'foo': 'bar'}),
]
```

- In this example, for a request to `/blog/2005/`, Django will call `views.year_archive(request, year=2005, foo='bar')`.
- If the views `year_archive` is
 - `def year_archive(request, year, **kwargs):`
 - ...

Including other URLconfs

- At any point, your url patterns can “include” other URLconf modules.

```
from django.urls import include, path
urlpatterns = [
    ....
    path('community/', include('aggregator.urls')),
    path('contact/', include('contact.urls')),
    ....
]
```

- Whenever Django encounters `include()`, it chops off whatever part of the URL matched up to that point and sends the remaining string to the included URLconf for further processing.

Views

Writing view

- View is simply a Python function that takes a Web request and returns a Web response
- This response can be the HTML contents of a Web page, or a redirect, or an XML document, or an image . . . or anything, really.
- The view itself contains whatever arbitrary logic is necessary to return that response. It's step through this code one line at a time:
- **Example:**
 - we define a function called `current_datetime`. This is the view function. Each view function takes an `HttpRequest` object as its first parameter, which is typically named `request`.
 - The view returns an `HttpResponse` object that contains the generated response.

```
from django.http import HttpResponse
import datetime

def current_datetime(request):
    now = datetime.datetime.now()
    html = "<html><body>It is now %s.</body></html>" % now
    return HttpResponse(html)
```

Writing view

- Django uses request and response objects to pass state through the system.
- When a page is requested, Django creates an `HttpRequest` object that contains metadata about the request. `HttpRequest` objects are created automatically by Django.
- Then Django loads the appropriate view, passing the `HttpRequest` as the first argument to the view function.
- Each view is responsible for returning an `HttpResponse` object.

```
>>> from django.http import HttpResponse
>>> response = HttpResponse("Here's the text of the Web page.")
>>> response = HttpResponse("Text only, please.", content_type="text/plain")
```

```
>>> response = HttpResponse()
>>> response.write("<p>Here's the text of the Web page.</p>")
>>> response.write("<p>Here's another paragraph.</p>")
```


Views using HttpResponseRedirect

```
# import Http Response from django
from django.http import HttpResponseRedirect,
From datetime import datetime
# create a function

def g_view(request):
    # fetch date and time
    now = datetime.datetime.now()
    # convert to string
    html = "Time is {}".format(now)
    # return response
    return HttpResponseRedirect(html)
```

Django shortcut functions

Render

Redirect

Django shortcut functions - Render

- `render(request, template_name, context=None, content_type=None, status=None, using=None)`
 - Renders a given template with a given context dictionary and returns an `HttpResponse` object with that rendered text.
 - Render combines a template + data and returns an `HttpResponse`: load template , fill it with data and create response
- **Two mandatory arguments :**
 - the `request`: the request object used to generate this response.
 - and `template_name` parameters.
- **Optional arguments**
 - **Status:** HTTP status code
 - `status=200` # OK (default)
 - `status=404` # Not Found
 - `status=403` # Forbidden
 - `status=500` # Server Error
 - Example use: `return render(request, "404.html", status=404)`

Django shortcut functions - Render

- **Optional arguments**
- **Context**: a dictionary of values to add to the template context. By default, this is an empty dictionary. Example:

```
{  
    "year": 2005,  
    "posts": posts  
}
```

In template:

```
{{ year }}  
{% for post in posts %}
```
- **Content_type**: the MIME type to use for the resulting document.
 - Sets the HTTP Content-Type header defaults to `'text/html'`.
- **Using**: the NAME of a template engine to use for loading the template. If None, Django uses the default engine.

Django shortcut functions - Render

- Django supports all types of HTML forms and rendering data from them to a view for processing using various logical operations.
- **Home.html**

```
<form action = "" method = "get">  
    <label for="your_name">Your name: </label>  
    <input id="your_name" type="text" name="your_name">  
    <input type="submit" value="OK">  
</form>
```

- **Views.py**

```
from django.shortcuts import render  
  
# Create your views here.  
def home_view(request):  
  
    # logic of view will be implemented here  
    return render(request, "home.html")
```


Django shortcut functions - Redirect

- `redirect(to, *args, permanent=False, **kwargs)`
- Returns an `HttpResponseRedirect` to the appropriate URL for the arguments passed.

```
# views.py
```

```
from django.shortcuts import redirect
```

```
def redirect_view(request):
```

```
    response = redirect('/redirect-success/')
```

```
    return response
```

- A view returning a redirect has to be added to your `urls.py`, like any other view:

```
# urls.py
```

```
from django.urls import path
```

```
from .views import redirect_view
```

```
urlpatterns = [
```

```
    path('/redirect/', redirect_view)
```

```
    # ... more URL patterns here
```

```
]
```


Django shortcut functions - Redirect

- **Passing URL**

```
def my_view(request):  
    ...  
    return redirect('/some/url/')  
    return redirect('https://example.com/')
```

- By passing the name of a **view** and optionally some positional or keyword arguments.

```
def my_view(request):  
    ...  
    return redirect('some-view-name', foo='bar')
```

```
path('example/<str:foo>/', views.some_view, name='some-view-name')
```

Django shortcut functions - Redirect

- **Why Redirect**
- When you are not logged-in and request a URL that requires authentication, like the Django admin, Django redirects you to the login page.
- When you log in successfully, Django redirects you to the URL you requested originally.
- When you change your password using the Django admin, you are redirected to a page that indicates that the change was successful.
- When you create an object in the Django admin, Django redirects you to the object list.
- When you type a short URL into the address bar of your browser and are then redirected to a page with a long, unwieldy URL.
- In other cases, redirects are an essential instrument to guide the user through a web application. After performing some kind of operation with side effects, like creating or deleting an object, it's a best practice to redirect to another URL to prevent accidentally performing the operation twice.

Django shortcut functions - Redirect

```
from django import forms
from django.http import HttpResponseRedirect
from django.shortcuts import redirect, render

def send_message(name, message):
    # Code for actually sending the message goes here

class ContactForm(forms.Form):
    name = forms.CharField()
    message = forms.CharField(widget=forms.Textarea)

def contact_view(request):
    # The request method 'POST' indicates
    # that the form was submitted
    if request.method == 'POST': # 1
        # Create a form instance with the submitted data
        form = ContactForm(request.POST) # 2
        # Validate the form
```


Django shortcut functions - Redirect

```
if form.is_valid(): # 3
    # If the form is valid, perform some kind of
    # operation, for example sending a message
    send_message(
        form.cleaned_data['name'],
        form.cleaned_data['message']
    )
    # After the operation was successful,
    # redirect to some other page
    return redirect('/success/') # 4
else: # 5
    # Create an empty form instance
    form = ContactForm()

return render(request, 'contact_form.html', {'form': form})
```

Django shortcut functions - Redirect

- If the view is called with a POST request, the POST data is used to instantiate a `ContactForm` object.
- If the form is valid, the form data is passed to `send_message()`. This function is not relevant in this context and therefore not shown here.
- After sending the message, the view returns a redirect to the URL `/success/`.
- If the view receives a GET request (or, to be precise, any kind of request that is not a POST request), it creates an instance of `ContactForm` and uses `django.shortcuts.render()` to render the `contact_form.html` template.
- If the user now hits reload, only the `/success/` URL is reloaded. Without the redirect, reloading the page would re-submit the form and send another message.

Ressources

- **Python regular expression:**
<http://docs.python.org/library/re.html>
- **URL Dispatcher:**
<https://docs.djangoproject.com/en/3.2/topics/http/urls/>
- **Writing Views:**
<https://docs.djangoproject.com/en/dev/topics/http/views/>
- **Request and response objects https:**
<https://docs.djangoproject.com/en/3.2/ref/request-response/>
- **Render [to response] function:**
<https://docs.djangoproject.com/en/dev/topics/http/shortcuts/>